VPCS: Verifiable Query Scheme for Privacy-preserving Constrained Shortest Path over Encrypted Graph Data

Shiyun He[a], Hongfa Ding*[,a,b,c], Tian Tian[a], Hai Liu[a], Heling Jiang[a]
[a] Guizhou University of Finance and Economics, China
[b] Guizhou University, China
[c] Data Security Certification (Guizhou) Co.,Ltd,China

2024 IEEE International Conference on Web Services (ICWS) | 979-8-3503-6855-0/24/$31.00 ©2024 IEEE | DOI: 10.1109/ICWS62655.2024.00145

***Abstract*—Currently, massive graph data, including social networks and biological proteins, is extensively utilized and contains a substantial amount of sensitive information. As cloud computing advances, graph data owners are increasingly inclined to outsource their large-scale graph data to cloud servers for diverse graph data query services. However, the seemingly limitless storage and computing capabilities present both opportunities and privacy challenges that are difficult to address. Recent research has proposed various schemes for querying outsourced graph data in an encrypted state. Unfortunately, many of these schemes fail to guarantee the correctness of query results under malicious models and may inadvertently disclose sensitive information within the graph data. Constrained shortest path (CSP) queries aim to find the shortest path between two vertices while adhering to specific threshold constraints. In this work, we propose a robust verifiable query scheme called VPCS that ensures privacy-preserving CSP queries on encrypted graphs. Our scheme achieves accurate and verifiable results while protecting the privacy of critical graph data information, with the exception of the number of vertices. Extensive experiments using real-world data sets validate the effectiveness of our scheme.**

***Index Terms*—Encrypted Graph, Verifiable Query, Constrained Shortest Path, Outsourcing**

I. Introduction

Graph data inherently presents entities and their associated features in both information and physical worlds. With the advancements in information technology, graph data has received wide attention from academia and industry. Various domains generate massive amounts of graph data, such as social media, scientific citations, email systems, and biological molecules. The collection, storage, mining, analysis, and utilization of these massive graph data often exceed the capabilities of the data owners. Therefore, it has become common practice to outsource or delegate the mining and analysis of graph data to cloud computing or third-party organizations. However, graph data in an outsourcing computation scenario faces risks of privacy and trade secret leakage, as it contains a significant amount of sensitive information [1], [2], such as personal identities, political affiliations, and geographical locations. In addition, several tools have emerged to address the analysis and processing of massive graphs, reflecting the widespread adoption of the technology, including GraphLab [3], TurboGraph++ [4] and GraphBase [5].

Compared to the shortest path, the Constrained Shortest Path (CSP) finds more applications in real-life scenarios. For instance, consider a transportation network where a delivery company needs to find the shortest route between two cities. By utilizing the shortest distance query, the company can minimize the costs such as money, travel time, and distance. This enables efficient planning of delivery routes, reducing fuel consumption, and improving overall logistics operations. However, there are always many constrained conditions, such as delivery time or limited budget. Thus, CSP plays a more important role in optimizing the transportation network. Several proposed solutions address the problem of approximate or exact CSP queries on plaintext graphs [6]–[8]. However, these solutions fail to guarantee the privacy of the graph data. Conversely, some solutions addressing CSP queries on encrypted graphs do not provide query users with the ability to verify the correctness of the query results [9]–[11]. Even various techniques, such as merkle hash tree [12], zero-knowledge proof [13], [14], and stream verification models [15] are adopted to verifiable computing schemes, but these methods are not suitable for CSP query verification on encrypted graph data. Consequently, designing efficient and secure verifiable computation schemes on encrypted graph data remains an urgent problem to be addressed.

To this end, we suggest VPCS, a scheme for efficiently and verifiably querying double-CSP over large-scale encrypted graph data. In this work, we make use of dummy edges to create a comprehensive outsourced graph data structure. Then we employ bilinear mapping and various cryptographic techniques such as secure multiparty computing, Diffe-Hellman and zero-knowledge proof to develop a Verifiable CSP query framework VPCS. This framework is designed to verify the correctness and integrity of the results provided by the outsourced server. In order to facilitate CSP querying, we utilize the Threshold Paillier Cryptosystem(TPC), which supports homomorphic addition operations, to define protocols such as Secure Sharing Comparison(SSC) and Secure Minimum(SM). By using VPCS, we can ensure the privacy of the graph data

This work is supported by NSFC (No.62002080), the Natural Science Researching Program of D.o.E. of Guizhou (No. Qian Edu. Tech. [2023] 065, Qian Edu. Tech. [2023]014), the China Postdoctoral Science Foundation (No. 2020M673584XB) and the Science and Technology Program of Guanshanhu District of Guiyang (No. Guan-Tech-Contract [2023]16).
*Corresponding author: Hongfa Ding (hfding@mail.gufe.edu.cn)

2836-3868/24/$31.00 ©2024 IEEE
DOI

while it is outsourced by the graph data owner. Additionally, the querying user is able to leverage proof information to verify the correctness and integrity of the query results. The key contributions of this work can be summarized as follows.

1) We propose the first graph ciphertext outsourcing system framework that supports privacy-preserving verifiable CSP queries. In our system framework, the sensitive information contained in the graph data is protected through the use of graph ciphertexts. These ciphertexts are generated by applying an encryption scheme called TPC that is an extension of the Paillier cryptosystem that retains the semantic security and additive homomorphism properties, which can ensures the protection of sensitive information.
2) We suggest VPCS, which ensures the protection of graph data privacy while enabling query users to verify the correctness of the obtained query results. This scheme provides a robust solution that allows users to validate the accuracy of the results while preserving the privacy of the underlying graph data.
3) We propose a novel zero-knowledge proof protocol to assist query users in verifying the correctness of their query results when the encrypted graph data is outsourced to a potentially malicious Computation Server(CS); This ensures that the query results are valid without revealing any sensitive information.
4) We conduct a comprehensive security analysis of VPCS, proving its resilience against Adaptive Chosen Query and Verification attack(IND-CQVA). Additionally, we perform extensive theoretical analysis and experimental evaluations using real-world data sets, demonstrating the practical advantages and effectiveness of VPCS.

II. RELATED WORK

Graph data has seen a surge in its outsourcing to cloud servers for storage and querying operations. Graph data outsourcing encompasses both plaintext [16]–[18] and encrypted [19]–[22]graph data for subgraph matching, shortest path querying, attribute querying, and Graphs' Intersection and Union. Here, we just focus on the work related to privacy preserving and verification.

After the initial work of structured encryption and encrypted graph data outsourcing by Chase and Kamara [23], various schemes for privacy preserving of graph computation have been proposed. Balanton et al. [24] designed oblivious algorithms for several classical graph problems, including breadth-first search, single-source shortest paths, minimum spanning tree, and maximum flow. Chang et al. [19] proposed a k-isomorphism based privacy-preserving subgraph matching scheme for large-scale graph data. Xu et al. [20] presented a privacy-preserving subgraph matching scheme based on secure multi-party computation. Suntaxi et al. [21] presented a scale-free graph query processing model focused on neighbor and adjacency queries by introducing the concept of confidentiality for graph-structured data and bucket number models. All these works can preserve the privacy of the outsourced graph data while providing different graph querying services. Among these services, shortest distance and path querying are the fundamental operations of graph computing, and there is an urge for privacy-preserving.

For the shortest path, numerous approaches to achieving approximate or accurate computation of the shortest path have been proposed to meet the diverse requirements of outsourcing computations on encrypted graph data [24]–[26]. After the original privacy preserving research about encrypted shortest path computing by Balanton et al. [24], Aly et al. [25] proposed the first special protocol for computing shortest paths by using black-box functions in a general multi-party computation environment. However, this protocol is only secure when participants are honest. Zhu et al. [27] proposed the Shortest Path Search for Synonymous Queries(SPSQ) solution, which supports synonym queries on encrypted graph data in cloud computing, utilizing stem extraction algorithms and encryption mechanisms. Lu et al. [28] introduced an online search method with pre-processed indexing for approximating point-to-point shortest paths. Liu et al. [26] proposed a secure order-preserving encryption scheme to perform approximate shortest distance queries by two-hop cover labeling. However, none of these schemes supports Constrained Shortest Distance (CSD) or CSP query.

To query CSD over encrypted graph data, Shen et al. [29] presented Connor, a graph-encryption scheme that supports approximate CSD query by employing a tree-based comparison protocol and Somewhat Homomorphic Encryption(SWHE). Subsequently, the same team [30] introduced Acro, another graph-encryption scheme designed to achieve high query accuracy for α-approximate CSD query. However, both of these schemes [29], [30] expose sensitive information such as out-degrees, in-degrees, and paths of two query vertices due to the Two-Hop Cover Lable(2HCL) structure. Zhang et al. [10] proposed a TPC based scheme called PGAS for privacy preserving accurate CSD querying by employing 2HCL and Pseudo-Random Function(PRF). But it still leaks the topological information by 2HCL. Wang et al. [11] employed TPC and virtual edges to propose a stronger privacy-preserving CSD query scheme called SPCS, they encrypt the edges and cost values in a homomorphic way and hide the the topological information by virtual edges, thus the sensitive information is protected while querying. However, these approaches [10], [11], [29], [30] just provide CSD query rather than a verification of the result for users.

To verify the outsourced result of graph computing, Fan et al. [17] applied the Merkle tree to authenticate the subgraph query service of outsourced graph databases, which was inspired by verifiable outsource computation [31]. After that, Peng et al. [16] proposed a scheme for efficient verification of subgraph similarity on outsourced graph data using metric indexing and a metric tree. Subsequently, Chen et al. [18] utilized Merkle Hash Tree to develop a verification scheme for sub-graph querying over social networks. With the advancement of verifiable computation, methods for verifiable outsourcing computation on encrypted graph data have

been gradually proposed. Zhu et al. [32] presented a verifiable outsourcing computation scheme for subgraph matching queries that allows for public and designated verification by utilizing a new cryptographic primitive called Boneh-Goh-Nissim cryptosystem. Zuo et al. [33] proposed a verifiable and privacy preserving scheme for graph data intersection querying in social network datasets based on a cryptographic accumulator. Yao et al. [12] proposed three schemes for verifiable single-attribute queries on outsourced social graphs by using cryptographic auxiliary information and a Merkle hash tree. However, these schemes just perform verification of queries for graphs such as subgraph matching, intersection, and attribute, not for verifiable CSP queries.

As mentioned above, there are various outsourcing schemes for encrypted graph data. Although some schemes provide CSD queries and some are verifiable solutions, they are not suitable for CSP querying. Thus, there still is an urgent requirement for a verifiable query scheme for privacy-preserving CSP over encrypted graph data.

To this end, we propose a novel scheme for secure and verifiable CSP querying over encrypted graph data based on TPC and zero knowledge. In this scheme, we employ homomorphism encryption and virtual edge construction to protect both private attributes and the topological structure of graph data. Then, in the adjacency matrix, we use the feature that the row vector and column vector of the vertex can be regarded as the out-degree set and in-degree set of the vertex to facilitate our calculation of CSP. We utilize the TPC to generate ciphertext and bilinear mapping-based zero-knowledge proof protocol to perform secure and verifiable CSP queries on encrypted data.

III. THRESHOLD PAILLIER CRYPTOSYSTEM

In this section, we provide the background and preliminary concepts for VPCS.

Definition 1 (Threshold Paillier Cryptosystem(TPC) [34]): TPC is a cryptographic scheme based on the Paillier encryption algorithm with thresholds. It can defined by the tuple of algorithms $TPC(KeyGen, Enc, Dec, Spkey, Pdec, Cdec)$.

$KeyGen(\xi)$. Choose a security parameter ξ and two large prime numbers p and q, such that $|p| = |q| = \xi$. Due to the properties of secure primes, there exit two secure primes p' and q' such that $p = 2p' + 1$, $q = 2q' + 1$. Then, we obtain the values of $N = pq$, $g = 1 + N$ and $\lambda = lcm(p-1, q-1)/2$. Finally, the public key is $pk = N$, and the private key is $sk = \lambda$.

$Enc(m, N)$. Given a plaintext $m \in \mathbb{Z}_N$, by selecting a random number $r \in \mathbb{Z}$, the corresponding ciphertext $[m]$ is obtained by

$$[m] = g^m \cdot r^N mod N^2 = (1 + mN) \cdot r^N mod N^2. \quad (1)$$

$Dec([m], sk)$. Due to the properties of the encryption scheme. To decrypt the ciphertext $[m]$ by sk, we can utilize the following equations.

$$[m]^\lambda = r^{\lambda N}(1 + mN\lambda) mod N^2 = (1 + mN)\lambda. \quad (2)$$

Additionally, it is given that $gcd(N, \lambda) = 1$. Therefore, the plaintext m can be obtained by

$$m = L([m]^\lambda mod N^2)\lambda^{-1} mod N, \quad (3)$$

where $L(x) = \frac{x-1}{N}$.

$Spkey(sk)$. By randomly selecting δ that satisfies $\delta \equiv 0 mod \lambda$ and $\delta \equiv 1 mod N^2$, where $gcd(N^2, \lambda) = 1$, we can define a polynomial $f(x) = \delta + \sum_{i=1}^{k-1} r_i \cdot x^i$, and $r_1, \cdots, r_{k-1} \in \mathbb{Z}^*_{\lambda N}$ are $k-1$ random numbers. Let $\beta_1, \cdots, \beta_n \in \mathbb{Z}^*_{\lambda N^2}$ be no-zero elements known to all n participants. Finally, we can define $sk_i = f(\beta_i)$ and send it to the i-th participant.

$Pdec([m], sk_i)$. The i-th participant computes a partial private key decryption based on his own private key $sk_i = f(\beta_i)$ and the ciphertext$[m]$ as

$$PD_i = [m]^{f(\beta_i)} mod N^2. \quad (4)$$

$Cdec(PD, \beta)$. Given all the partial decryption $PD = \{PD_1, \cdots, PD_d\}$, where $d \geq k$ and the non-zero elements $\beta_1, \cdots, \beta_n \in \mathbb{Z}^*_{\lambda N^2}$, and selecting k distinct numbers as $U = \{u_1, \cdots, u_k\}$, where $1 \leq u_i \leq d$. Then

$$T = \prod_{i=1}^{k} (PD_i)^{\triangle(u_i, 0)} mod N^2. \quad (5)$$

Here, $\triangle(u_i, x) = \prod_{j \in U, j \neq s_i} \frac{x - \beta}{\beta_{u_i} - \beta_j}$. Finally, the decrypted result is $m = L(T)$.

Furthermore, in $TPC(KeyGen, Enc, Dec, Spkey, Pdec, Cdec)$, for any $m \in \mathbb{Z}_N$, we have

$$[m]^{N-1} = (1 + (N-1)m \cdot N) \cdot r^{N-1 \cdot N} mod N^2 = [-m]. \quad (6)$$

Additionally, for the same public key pk, the system satisfies additive homomorphism, i.e., $Add([m1_1], [m_2]) = Enc(m_1 + m_2)$.

IV. THE FRAMEWORK AND ALGORITHMS OF VPCS

A. The Framework of VPCS

The owner GO of graph data $G = (V, E)$ wants to outsource G to the Computation Server CS, and CS cooperates with a semi-honest server proxy Proxy to provide verifiable CSP querying for Query Users QU. The framework overview of VPCS is shown in Fig. 1. There are four participants in this framework: GO, CS, Proxy, and QU, and these participants interact with each other.

GO. GO generates the keys of VPCS, encrypts the raw graph data, and generates auxiliary information for verification. Firstly, GO executes algorithm $KeyGen$ to generate a threshold Paillier key pair (pk, sk), a PRF key k and PRF $\mathcal{F}_{\mathcal{PRF}}$ and algorithm $Spkey()$ to divide private key sk into sk_1 and sk_2 for enabling secure share comparison protocol SSC and Secure Minimum value algorithm SM between CS and Proxy. In there, GO send sk_1 to Proxy, share sk_2 to CS, and send the PRF $\mathcal{F}_{\mathcal{PRF}}$, k and sk to QU through the security channel; Secondly, GO executes algorithm $GraphEnc$ on G by using public key pk, k, and outputs ciphertext $[G]$ with auxiliary information $g^{p(S)}$, g, S and H. $[G]$ is outsourced

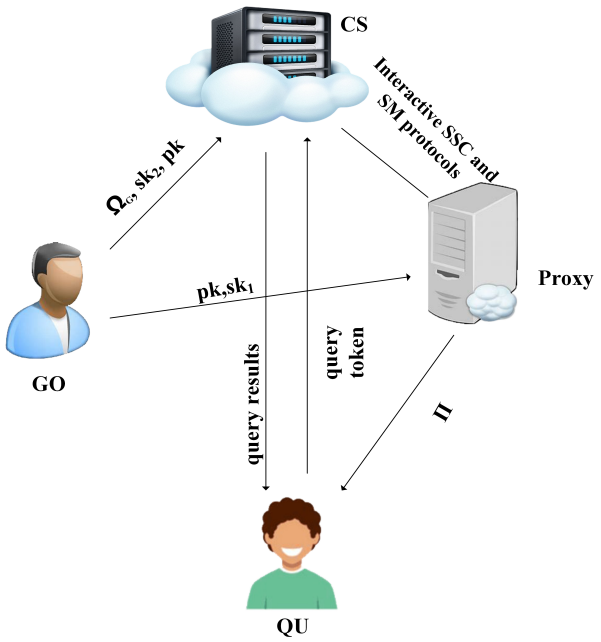


Fig. 1: System model of our scheme.

to CS, $g^{p(S)}$, g and S are sent to all participants, and H is shared with CS and Proxy. Note that $GraphEnc$ can perform a line-by-line encryption process on the adjacency matrix of G. Thus, the computation requirement of GO is not too large.

CS. CS stores $[G]$, $g^{p(S)}$, g, S, and H received from GO, computes the CSP query, and verification cooperated with Proxy for QU's queries. Firstly, CS stores $[G]$, $g^{p(S)}$, g, S and H, $[G]$ is stored as an adjacency matrix, where each element represents a ciphertext triplet ([d], [c], [t]). These triplets correspond to the encrypted values of the distance, cost, and time between every pair of vertices. Secondly, after receiving a query token q from QU, CS executes algorithm $CSPQuery$ cooperated with Proxy and outputs the shortest path $dist_{path}$ and shortest distance $dist_p$. In VPCS, we assume CS is malicious, and CS may tamper, alter, and manipulate the data during storage and computing.

Proxy. Proxy is semi-honest, stores $g^{p(S)}$, g, S, and H received from GO, and computes the CSP query and verification cooperated with CS for QU's queries. Proxy executes algorithm $CSPQuery$ cooperated with CS and outputs verifiable proof π.

QU. QU generates query tokens for CSP queries and decrypts and verifies the queried results returned from CS and Proxy. Firstly, QU executes algorithm $TokenGen()$ to generate a token TQ_{st} if a CSP query from s to t is needed. Secondly, after receiving queried result $dist_{path}$, $dist_p$ and π from CS and Proxy, QU executes algorithm $Verify$ and Dec to verify and decrypt the queried result. Note that QU also can be played by GO, while it cannot collude with CS and Proxy.

B. Components and Workflow of VPCS

As shown in Sec.IV-A, there are four participants in VPCS, and they need to interact with each other to achieve verifiable and privacy-preserving CSP querying. And here, the algorithm components and workflow of VPCS are detailed.

There are five algorithms and three protocols in VPCS, and the algorithms are $KeyGen$, $GraphEnc$, $TokenGen$, $Verify$, and $CSPDec$. The protocols are $CSPQuery$, SSC and SM. And VPCS can be regarded as three parts, namely Initialization, Encryption and Outsourcing, and CSP Service Providing. The former two parts just need to be performed one time, and the last part can work thousands of times.

Firstly, in the phase of initialization, GO generates and splits the keys for all the participants. Secondly, in the phase of encryption and outsourcing, GO encrypts the graph data and generates some auxiliary information. Then the encrypted data is outsourced to CS, and the auxiliary information is distributed to all relative participants. Thirdly, in the phase of CSP service providing, QU generates a querying token if a CSP query is needed and sends this token to CS and Proxy. Then, CS and Proxy query the CSP result and generate proofs of the CSP result interactively. Then, QU verifies and decrypts the CSP result with the proofs and private key.

1) Initialization, Encryption, and Outsourcing: In initialization, GO executes $KeyGen$ with security parameter ξ as input, and outputs (pk, sk). And GO executes $Spkey$ to split sk into sk_1 and sk_2, $KeyGen$ and $Spkey$ follow Def.1. GO chooses an arbitrary reversible PRF $\mathcal{F}_{\mathcal{PRF}}$ and generates symmetric key k.

In Encryption and Outsourcing, GO executes $GraphEnc$ with G, pk, k and $\mathcal{F}_{\mathcal{PRF}}$ as inputs, and outputs $[G]$, g, S and G. Then GO outsources $[G]$ to CS and sends auxiliary information g, S, and G to other relevant participants. $GraphEnc$ is detailed in Alg. 1. In Alg. 1, G is presented as adjacency matrix $A = (a_{i,j})_{n\times n}$, and the values of $a_{i,j} = (d_{i,j}, c_{i,j}, t_{i,j})$ are the actual values of distance, cost and time between v_i and v_j, and the values are infinity if there is no edge between v_i and v_j. Here, we employ a virtual edge φ and set considerable values (i.e., values greater than the biggest actual value.) instead of infinitive values.

Not that, no one except GO can be aware of which edges are dummy edges φ due to the indistinguishability of TPC. This ensures the non-disclosure of the topological structure of G. Among the outputs of $GraphEnc$, $g^{p(S)}$, g, and S are the auxiliary information for verifying the CSP result, and H is the auxiliary information for CSP querying.

2) CSP Service Providing: In this phase, there are three steps as discussed above. Firstly, QU generates a query token by $TokenGen$ as a query $q_{st} = (s, t, \theta_1, \theta_2)$ need. q_{st} means a CSP query from s to t with constraints of cost θ_1 and time θ_2. $TokenGen$ is detailed as Alg. 2 with q_{st}, k, S, pk and g as inputs, and token TQ_{st} as output. TQ_{st} is sent to CS and Proxy for CSP querying. Note that α is reserved privately by QU as g^{α} and $g^{\alpha \cdot S}$ play critical roles in verifying the correctness and integrity of the CSP result by QU himself.

Algorithm 1 GraphEnc

Input: $G = (V, E)$, k, $\mathcal{F}_{\mathcal{PRF}}$ and pk.
Output: $[A]_{n\times n}$, H, S, g and $g^{p(S)}$.

1: $V = (v_1, \ldots, v_n)$ stores all vertices of G;
2: $H = \mathcal{F}_{\mathcal{PRF}}(k, v_i)_{i=1}^{n} = [h_1, \ldots, h_n]$;
3: Encrypt $[A]_{n\times n}$ by Enc of TPC, where $[a_{ij}] = Enc(a_{ij}, pk)$;
4: Generate a random set $S = \{s_1, \ldots, s_n\}$ with n elements;
5: Compute $g^{p(S)} = g^{\sum_{i=1}^{n} s_i \times h_i}$;
6: Publish $[A]_{n\times n}$, $g^{p(S)}$, g and S to all participants, share H with CS and Proxy.

Algorithm 2 TokenGen

Input: $q_{st} = (s, t, \theta_1, \theta_2)$, k, S, pk and g.
Output: TQ_{st}.

1: Compute $h_s = \mathcal{F}_{\mathcal{PRF}}(k, s)$ and $h_t = \mathcal{F}_{\mathcal{PRF}}(k, t)$;
2: Compute $\Theta_1 = [\theta_1]$ and $\Theta_2 = [\theta_2]$ by Enc of TCP, where $[\theta] = Enc(\theta, pk)$;
3: Generate a random number α and compute g^{α},$g^{\alpha \cdot S} = \{g^{\alpha \cdot s_1}, \ldots, g^{\alpha \cdot s_n}\}$;
4: Set $TQ_{st} = (h_s, h_t, \Theta_1, \Theta_2, g^{\alpha}, g^{\alpha \cdot S})$.

Secondly, CS and Proxy search the CSP of QT_{st} by protocol CSPQuery interactively. CSPQuery is detailed as Proto. 1 with $[A]$, H, QT_{st} and sk_2 as CS's inputs, and H, QT_{st} and sk_2 as Proxy's inputs. Then, $dist_{path}$ and $dist_p$ are CS's outputs, and π is the Proxy's output. And as shown in Proto. 1, CSPQuery needs a secure sharing comparison SSC protocol and a Secure Minimum value SM algorithm to get the expected results. The details of SSC and SM will be presented in Sec. IV-C and Sec. IV-D, respectively. After calculating the CSD $dist_p = [d_{min}]$ and CSP $dist_{path}$, the Proxy will output a proof $\pi = \{g^{\delta p(S)}, g^{\delta \alpha p(S)}, g^{\delta p(S)'}\}$, where $g^{\delta p(S)} = \{g^{\sum_{i=1}^{n} s_i \times h_i}\}^{\delta}$, $g^{\delta \alpha p(S)} = \{\prod_{i=1}^{n}\{g^{\alpha s_i}\}^{h_i}\}^{\delta}$, $g^{\delta p(S)'} = \{g^{\sum_{i=1, i\neq min}^{n} s_i \times I_i} \cdot g^{h_{min}}\}^{\delta}$.

Thirdly, after receiving π from Proxy and $dist_{path}, dist_p$ from CS, QU verifies $dist_{path}, dist_p$ via π and α by algorithm $Verify$, and QU decrypts $dist_{path}, dist_p$ by algorithm $CSPDec$ if $Verify$ returns true. $Verify$ is detailed as Alg. 4 with π and α as inputs, and boolean value b as output. $CSPDec$ is detailed as Alg. 5 with $dist_{path}$, $dist_p$, b, g and sk as inputs, plaintext of CSP and CSD as outputs.

C. Secure Sharing Comparison Protocol

As mentioned above, when querying CSP over encrypted graph data, we need to compare two encrypted values $[x]$ and $[y]$ to determine which path or distance is shorter between two different edges. Here we present the SSC protocol inspired by the idea in [10], and our SSC protocol is different from the ones of Wang et al. [11]. The presented SSC protocol implements the comparison of two encrypted values based on private key splitting of TPC, and the SSC protocol is detailed as Proto. 2.

Protocol 1 CSPQuery

Input: CS: $[A]_{n\times n}$, $H = [h_1, \ldots, h_n]$, $TQ_{st} = (h_s, h_t, \Theta_1, \Theta_2, g^{\alpha}, g^{\alpha \cdot S})$ and sk_2; Proxy: sk_1, $H = [h_1, \ldots, h_n]$ and $TQ_{st} = (h_s, h_t, \Theta_1, \Theta_2, g^{\alpha}, g^{\alpha \cdot S})$
Output: CS: $dist_{path}$ and $dist_p$; Proxy: π.

CS:
Set the row vector $[A]_{h_s,\cdot}$ as $R_s = (r_{s,1}, \ldots, r_{s,n})$;
Set the column vector $[A]_{\cdot,h_t}$ as $C_t = (c_{t,1}, \ldots, c_{t,n})$;
Proxy:
for i=1 to n **do**
  Set both I and P as n elements with 0;
  Set $D = \{i \mid (r_{s,i} = \varphi) \cap (c_{t,i} = \varphi)\}$;
  if $i \in D$ **then**
    $I[i] = H[i]$;
  else
    $[p_i] = ([d_i], [c_i], [t_i])$, where $[d_i] = [d_{s,i}] \cdot [d_{t,i}]$, $[c_i] = [c_{s,i}] \cdot [c_{t,i}]$, $[t_i] = [t_{s,i}] \cdot [t_{t,i}]$;
    CS and **Proxy**:
    $a = SSC(Proxy : [c_i], \Theta_1, sk_1; CS : sk_2)$;
    $b = SSC(Proxy : [t_i], \Theta_2, sk_1; CS : sk_2)$;
    if $a = 0$ or $b = 0$ **then**
      $I[i] = H[i]$;
    else
      $P[i] = [p_i]$
    end if
  end if
end for
CS and **Proxy**:
$dist_p = [d_{\min}] = SM(P)$, where CS obtains $[d_{\min}]$ and also obtains the index i where the result is not the shortest path, and sets $I[i] = H[i]$;
Proxy:
$g^{\delta p(S)} = \{g^{\sum_{i=1}^{n} s_i \times h_i}\}^{\delta}$, $g^{\delta \alpha p(S)} = \{\prod_{i=1}^{n}\{g^{\alpha s_i}\}^{h_i}\}^{\delta}$, $g^{\delta p(S)'} = \{g^{\sum_{i=1, i\neq min}^{n} s_i \times I_i} \cdot g^{h_{min}}\}^{\delta}$.
Calculate $g^{\delta \alpha p(S)}$ and $g^{\delta \alpha p(S)'}$ using I and $[d_{min}]$, where the Proxy obtains a proof: $\pi = \{g^{\delta p(S)}, g^{\delta \alpha p(S)}, g^{\delta p(S)'}\}$.

In Proto. 2, the comparison is collaboratively conducted by Proxy and CS, ensuring a secure comparison protocol without compromising the complete decryption of the two ciphertests. Proto. 2 yields a Boolean value $u^* = 1$ to Proxy if plaintext $x < y$, otherwise, $x \geq y$. The data transmission between CS and Proxy occurs over a secure channel, ensuring the confidentiality of the communicated data.

Firstly, Proxy obscures the the equivalence relation between x and y (line 2), and conceals the relationship between x and y further with r_1, r_2 and b (line 3-9). Thus, CS cannot determine which of x or y is greater by decrypting $[z]$ without knowledge of b.

Here, if x and y are integers, we have

$$x_1 < y_1 \iff 2x + 1 < 2y \iff x < y \tag{7}$$

$$x_1 > y_1 \iff 2x + 1 > 2y \iff x \geq y \tag{8}$$

Protocol 2 SSC

Input: Proxy: $[x]$, $[y]$ and sk_1; CS: sk_2.
Output: Proxy: u^*.

Proxy:
$[x_1] = [x]^2[1] = [2x+1]$,$[y_1] = [y]^2 = [2y]$;
Randomly choose $r_1, r_2 \in \mathbb{Z}_N$;
Randomly choose a bit $b \in \{0,1\}$;
if $b = 1$ **then**
  $[z] = ([x_1][y_1]^{N-1})^{r_1} \cdot [r_2]$;
else
  $[z] = ([x_1]^{N-1}[y_1])^{r_1} \cdot [r_2]$;
end if
$z_1 = Pdec([z], sk_1)$;
send $[z], z_1$ to CS;
CS:
$z_2 = Pdec([z], sk_2)$;
$PD = \{z_1, z_2\}$;
$z = Cdec(PD)$;
if $|z| > |N|/2$ **then**
  $u' = 0$;
else
  $u' = 1$;
end if
send u' to Proxy;
Proxy:
if $b = 1$ **then**
  $u^* = u'$;
else
  $u^* = 1 - u'$.
end if

Algorithm 4 Verify

Input: The proof $\pi = \{g^{\delta p(S)}, g^{\delta \alpha p(S)}, g^{\delta p(S)'}\}$, α.
Output: boolean b.

$b = 0$;
if $g^{\delta p(S)} = g^{\delta p(S)'}$ **then**
  if $e(g^{\delta \alpha p(S)}, g) = e(g^{\delta p(S)}, g^{\alpha})$ **then**
    $b = 1$;
  end if
else
  $b = 0$.
end if

Secondly, CS helps Proxy by recovering the paintext z of $[z]$ (line 13-15), and computes the relationship between x and y (line 16-20), but he does not know which one is greater. Finally, Proxy computes the result u^* based on the chosen bit b (line 23-27).

D. Secure Minimum

As mentioned in Sec. IV-B, protocol CSPQuery also needs a protocol to select the minimum over encrypted candidate CSDs. Here, we present the Secure Minimum (SM) protocol based on the SSC protocol. The details of SM protocol are shown in Proto.3.

Algorithm 5 CSPDec

Input: b, g, sk, $dist_{path}$, and $dist_p$.
Output: plaintext of CSP and CSD (p, c).

if b=0 **then**
  return $(0, 0)$.
else
  $p = D_{PRF}(dist_{path}, k)$, $c = Dec(dist_p, sk)$, and D_{PRF} is the inversion of $\mathcal{F}_{\mathcal{PRF}}$.
end if

Protocol 3 SM

Input: Proxy: sk_1 and $P = \{[p_1], \ldots, [p_n]\}$; CS: sk_2.
Output: CS and Proxy: encrypted CSD $[d_{\min}]$.

Proxy:
Set $[d_{\min}] = [p_1]$;
for all $i = 2, 3, \cdots, n$ **do**
  if $p_i \neq= 0$ **then**
    CS and **Proxy**:
    $[d_{min}] = SSC(Proxy : [p_i], [d_{min}], sk_1; CS : sk_2)$;
  end if
end for

V. Correctness Analysis

As we utilize the TPC to encrypt the GO's raw graph, and the correctness of TPC has been proven by Liu et al. [34]. Thus, we just focus on the correctness analysis of the verification process in VPCS.

Theorem 1: If VPCS strictly ensures the non-disclosure of parameters a and b while ensuring the correctness of Bilinear Mapping(BM), it can accurately guarantee the privacy of GO's graph data without leakage and at the same time verify the correctness and integrity of QU's query results from CS and Proxy.

Proof 1: Leveraging BM, Bilinear Diffe-Hellman(BDH), and Deciaional Bilinear Diffie-Hellman(DBDH), the VPCS scheme enables verifiable CSP query verification, ensuring data privacy, result integrity, and correctness. The QU requests a query from the GO, who grants permission. The QU then generates a TQ_{st} and sends it to CS and Proxy. Then, QU will receive the result of CSP and a Proof $\pi = \{g^{\delta p(S)}$,$g^{\delta \alpha p(S)}$, $g^{\delta p(S)'}\}$ from CS and Proxy. QU will verify the CSP's integrity and correctness by π.

Using α and δ ensures privacy, preventing collusion and data forging. Thus, the above two parameters play a crucial role in CSP query and verification. δ limits QU's node information to the CSP, and α prevents GO and Proxy collusion as the BDH and DBDH.

QU verifies the shortest path computation by checking equations $g^{\delta p(S)} \stackrel{?}{=} g^{\delta p(S)'}$ and $e(g^{\delta \alpha p(S)}, g) \stackrel{?}{=} e(g^{\delta p(S)}, g^{\alpha})$ as the BM's characteristic.

Finally, by following this verification procedure based on the principles of BM, QU can effectively assess the correctness of the shortest path computation process performed by CS and Proxy. □

VI. SECURITY DEFINITION AND ANALYSIS

In this section, we conducted a detailed analysis of the security and privacy of the proposed VPCS. We began by formally defining the leakage function and presented proof of the security of VPCS under the IND-CQVA-security model, which is defined as Def. VI-B. Furthermore, we highlighted that VPCS achieves advanced privacy protection for verifiable queries compared to the approaches described in [10], [11], [29].

A. Leakage Function

1) Encryption leakage $\mathcal{L}_{Enc}$: The encryption leakage function $\mathcal{L}_{Enc}$ indicates the amount of privacy information that can be inferred from the encrypted adjacency matrix $[A]_{n\times n}$. In our proposed VPCS, the adversary can only obtain the number of vertices, denoted as n, from the encrypted adjacency matrix $[A]_{n\times n}$ of graph data G. Therefore, the form of the leakage function is $\mathcal{L}_{Enc}(G) = n$.

2) Query leakage $\mathcal{L}_{Query}$: For the query sequence $Q = \{(s_1, t_1, \theta_{1,1}, \theta_{1,2}), \cdots, (s_n, t_n, \theta_{n,1}, \theta_{n,2})\}$, during the query token generation phase, the QU generates values for the query vertices using $\mathcal{F}_{\mathcal{PRF}}$ and encrypts the cost and time constraints using TPC. Due to the semantic security property of TPC, there is no probabilistic polynomial-time (PPT) algorithm that can determine which queries in the query sequence Q have the same cost constraints. Therefore, $\mathcal{L}_{Query}(Q)$ only includes the queries in the query sequence that target the same pair of vertices. This can be expressed as $\mathcal{L}_{Query}(Q) = \{q_i | \exists q_j \in Q/q_i, H_{s_i} = H_{s_j} \vee H_{t_i} = H_{t_j} \vee H_{s_i} = H_{t_j} \vee H_{t_i} = H_{s_j}\}$.

B. Security Definition

In this section, we modify the security definitions proposed in [10], which is called IND-CQVA-security , and formalize it for our proposed graph outsourcing scheme VPCS.

Definition 2 (IND-CQVA-security): Let Ω=(KeyGen, GraphEnc, CSPQuery, Verify) denote an outsourcing system for encrypted graph data, and $\mathcal{L} = (\mathcal{L}_{Enc}, \mathcal{L}_{Query}, \mathcal{L}_{verify})$ be the leakage functions of Ω. Given a security parameter ξ, a semi-honest adversary $\mathcal{A}$, a challenger $\mathcal{C}$ and a simulator $\mathcal{S}$, we construct the following probabilistic experiment.

1) $Real_{\Omega,\mathcal{A}}(\xi)$:
   - The adversary $\mathcal{A}$ selects a graph G and transmits it to the challenger $\mathcal{C}$.
   - The challenger $\mathcal{C}$ runs $KeyGen(\xi)$ to generate a key pair (pk, sk), PRF key k, and a generator g. Then, $\mathcal{C}$ executes $GraphEnc(G, k, pk, g)$ to generate $[A]_{n\times n}$, $g^{p(s)}$, H and S, which are sent to $\mathcal{A}$.
   - The adversary $\mathcal{A}$ adaptively generates a polynomial sequence of shortest path queries $Q = \{(s_1, t_1, \theta_{1,1}, \theta_{1,2}), \cdots, (s_n, t_n, \theta_{n,1}, \theta_{n,2})\}$ and sends it to $\mathcal{C}$.
   - For each query $q_i = (s_i, t_i, \theta_{i,1}, \theta_{i,2})$, $\mathcal{C}$ runs $TokenGen(q_i, K, S, pk, g)$ to obtain a query token $TQ_{s_i t_i}$, and sends $TQ = \{TQ_{s_1 t_1}, \cdots, TQ_{s_n t_n}\}$ to $\mathcal{A}$.
   - The adversary $\mathcal{A}$ performs the shortest path query based on TQ using $CSPQuery$, and then verifies the integrity and correctness of the computed results by executing $verify$ in collaboration with $\mathcal{C}$.
   - $\mathcal{A}$ outputs a bit $b \in 0, 1$ as the result of this experiment.
2) $Ideal_{\Omega,\mathcal{A},\mathcal{S}}(\xi)$:
   - The adversary $\mathcal{A}$ selects a graph G and transmits it to the challenger $\mathcal{C}$.
   - $\mathcal{C}$ provides the output of leakage function $\mathcal{L}_{Enc}(G)$ to the simulator $\mathcal{S}$, and then $\mathcal{S}$ generates and sends $[A]_{n\times n}$ to $\mathcal{A}$.
   - The adversary $\mathcal{A}$ adaptively generates a polynomial sequence of shortest path queries $Q = \{(s_1, t_1, \theta_{1,1}, \theta_{1,2}), \cdots, (s_n, t_n, \theta_{n,1}, \theta_{n,2})\}$ and sends it to $\mathcal{C}$.
   - For each query $q_i = (s_i, t_i, \theta_{i,1}, \theta_{i,2})$, $\mathcal{C}$ sends the output value of the leakage function $\mathcal{L}_{Query}(Q)$ to $\mathcal{S}$, where Q refers to a series of queries, and $\mathcal{S}$ simulates the generation of query tokens $TQ = \{TQ_{s_1 t_1}, \cdots, TQ_{s_n t_n}\}$ to be sent to $\mathcal{A}$.
   - $\mathcal{A}$ executes $CSPQuery$, and $\mathcal{C}$ sends the value of the leakage function $\mathcal{L}_{verify}(\alpha)$ to $\mathcal{S}$, where α refers to the verification parameters used during a series of query processes. Then, simulates the execution of $verify$.
   - $\mathcal{A}$ outputs a bit $b \in 0, 1$ as the result of this experiment.

If for all PPT adversaries $\mathcal{A}$, there exists a PPT simulator $\mathcal{S}$ and a negligible function $negl(\xi)$, such that

$$\left|\Pr[Real_{\Omega,\mathcal{A}(\xi)} = 1] - \Pr[Ideal_{\Omega,\mathcal{A},\mathcal{S}(\xi)} = 1]\right| \leq negl(\xi).$$

Then, under this condition, the security of $\mathcal{L} = (\mathcal{L}_{Enc}, \mathcal{L}_{Query}, \mathcal{L}_{verify})$ is established against IND-CQVA attacks. This means that the scheme can withstand adaptive chosen query and verification attacks.

C. Security of VPCS

In this section, we will provide proof of the IND-CQVA-security of VPCS proposed in our work.

Theorem 2: If PRF and TPC are security, then VPCS is $\mathcal{L} = (\mathcal{L}_{Enc}, \mathcal{L}_{Query}, \mathcal{L}_{verify}) - security$ against adaptive chosen query and verify attack.

Proof 2: By utilizing the constructed simulator S and three leakage functions, namely $\mathcal{L}_{Enc}$, $\mathcal{L}_{Query}$ and $\mathcal{L}_{Verify}$, we simulate a ciphertext graph $[A^*]_{n\times n}$, a query token TQ^*, and a parameter α^*. If, for any PPT adversary $\mathcal{A}$, they are unable to distinguish between the Real and Ideal probability experiments, it can be concluded that the scheme achieves IND-CQVA-security.

Simulating $[A^*]_{n\times n}$: Firstly, the simulator S randomly generates a virtual PRF key k^* and a set of virtual vertices

TABLE I: Comparative Analysis of Verifiable Outsourcing Strategies

Scheme	Type	Method	Security
VQOS [12]	Attribute query	merkle hash tree	Collision-free
VPGI [33]	Intersection query	Pairing-based accumulator	Collision-free
VMAC [27]	subgraph matching	Generic accumulator	Collision-free
Our VPCS	double-CSP	MPC-ZKP	IND-CQVA Collision-free

$V^* = \{v_1^*, \cdots, v_n^*\}$. For each vertex, $\mathcal{S}$ computes the PRF value with k^*, resulting in $V_{k^*}^* = \{H(k^*, v_1^*), \cdots, H(k^* v_n^*)\}$. The distances and costs of the edges between vertices are replaced with random numbers. Subsequently, using the TPC, $\mathcal{S}$ encrypts each edge with the generated public key pk. Finally, $\mathcal{S}$ obtains a virtual ciphertext adjacency matrix $[A^*]_{n\times n}$.

Simulating TQ^*: For a given query sequence $Q = \{q_1, \cdots, q_n\}$ and its corresponding query token list $TQ = \{TQ_{s_1 t_1}, \cdots, TQ_{s_n t_n}\}$, and given a leakage function $\mathcal{L}_{Query}(q)$ for each query q_i, $\mathcal{S}$ proceeds to simulate the query token as follows: $\mathcal{S}$ first checks if the s_i and t_i values appearing in the query have been encountered before. If s_i has been encountered previously, $\mathcal{S}$ assigns H_{s_i} the same value as the previous occurrence; otherwise, $\mathcal{S}$ assigns $H_{s_i} = \eta$, η is a non-repeating random integer and this assigning stores the relationship between s_i and η. A similar process is carried out for t. Then, $\mathcal{S}$ randomly selects two values and encrypts them using TPC to simulate two ciphertext constraint conditions Θ_1 and Θ_2. Finally, the simulated query token TQ^* is obtained.

Simulating $verify^*$: $\mathcal{S}$ simulates an α and then randomly selects a generator $g \in \mathbb{G}_1$ and δ. Using the previously simulated pseudo-random function values $V_{k^*}^*$ of the vertices, $\mathcal{S}$ generates an array S^* with a length equal to $len(V_{k^*}^*) = len(S^*)$. Given a leakage function $\mathcal{L}_{verify}$, $\mathcal{S}$ computes a simulated proof. Finally, the $verify^*(proof^*)$ is obtained.

If PRF $\mathcal{F}_{\mathcal{PRF}}$ and TPC are secure, the simulated encrypted adjacency matrix $[A^*]_{n\times n}$, query token TQ^*, and verification $proof^*$ are indistinguishable from the real ones. As a result, no PPT adversary can distinguish between the real and ideal probabilistic experiments. Thus, we obtain

$$\left|\Pr[Real_{\Omega,\mathcal{A}(\xi)} = 1] - \Pr[Ideal_{\Omega,\mathcal{A},\mathcal{S}(\xi)} = 1]\right| \leq negl(\xi).$$

□

VII. PERFORMANCE ANALYSIS

A. Theoretical Analysis

As shown in Table II, it can be observed that Connor, PGAS, and SPCS support CSD queries. However, Connor only supports approximate queries, while PGAS and SPCS support accurate CSD queries. In contrast, our proposed VPCS scheme supports double-CSP queries with verification capability.

The efficiency of outsourcing computation for the shortest path in graphs is primarily influenced by the GraphEnc, TokeGen, and CSPquey algorithms. In Table II, we measure and compare the efficiency among Conor, PGAS, SPCS and VPCS based on computational and communication complexities. We evaluate computational complexity by comparing the number of modular exponential operations during the computation process. Additionally, we measure communication complexity by considering the number of ciphertexts exchanged between entities. Here, m represents the number of edges in the graph data being processed, n represents the number of candidate paths between two queried vertices, and $d\theta$ represents the depth of the binary tree used for comparing during CSPQuery algorithm in Connor.

From Tables II, it is evident that compared to Connor, PGAS and SPCS, VPCS achieves verifiable double-CSP queries without any additional computational or communication complexity. To further protect the topology information of the graph, it employs dummy edges to obfuscate the structure, which is also a contributing factor to the relatively high time complexity during the encryption phase. Furthermore, as demonstrated in Table I, our VPCS scheme exhibits IND-CQVA-security and Collision-free Verification Scheme, surpassing other verification methods.

B. Experimental Analysis

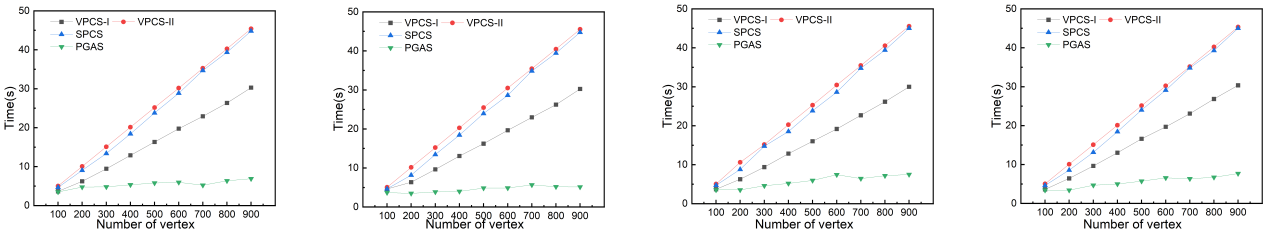
To further evaluate the efficiency of VPCS, we conducted experiments using publicly available real-world graph datasets and compared the performances among PGAS, SPCS and VPCS with CSD query. VPCS-I donates a single-CSD query of VPCS and is used for comparing with PGAS, and VPCS-II donates a double-CSD query of VPCS and is used for comparing with SPCS. All of PGAS, SPCS and VPCS are implemented by Python. The experiments are conducted on Ubuntu 18.04 Linux X86-64 with CPU HyGon 7381 and 32GB RAM.

1) Datasets: We employed publicly accessible real-world graph datasets obtained from the SNAP website. The specific details of the datasets are outlined in Tab. III. For convenience, we randomly select subgraphs from these five graphs and assign random values for distances, costs, and time to each edge.

2) Experimental result: Fig. 2 presents the query time of the four schemes across four different datasets. For single-CSD queries, PGAS has a shorter query time compared to VPCS-I due to the encryption of the outsourced graph data using the structure of vertex out-degree labels and in-degree labels in PGAS. However, this scheme leaks the topological structure of the graph data to the cloud server. Additionally, during queries, PGAS also leaks the number of paths between two query vertices. For double-CSD queries, the query time of SPCS is similar to that of VPCS-II as both schemes perform double-CSD queries on the encrypted adjacency matrix. However, in the case of dummy edges, SPCS does not encrypt them; instead, it constructs a random number within a specific range,

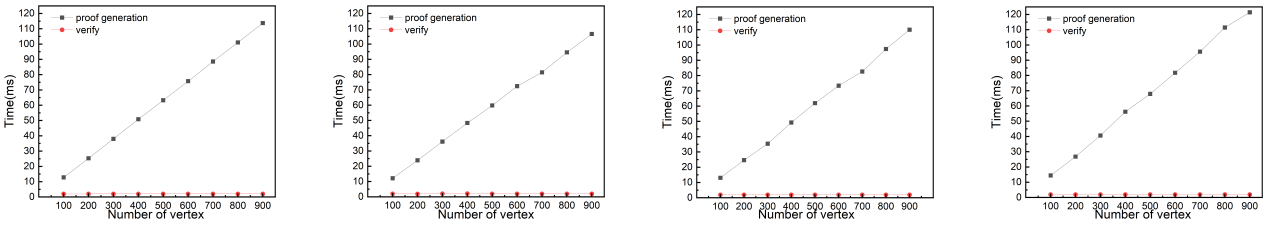
TABLE II: Efficiency Comparison among Different Schemes of Outsourcing Computation over Graph Data

Scheme	Query Type	Computation CPX	Communication CPX
Connor [30]	approximate single-CSD	GraphEnc $O(m)$ TokenGen $O(2^{d\theta})$ DistQuery $O(nd\theta)$	TokenGen $O(2^{d\theta})$ Distance $O(1)$
PGAS [10]	single-CSD	GraphEnc $O(m)$ TokenGen $O(1)$ DistQuery $O(n)$	TokenGen $O(1)$ Distance $O(1)$
SPCS [11]	double-CSD	GraphEnc $O(m)$ TokenGen $O(1)$ DistQuery $O(n)$	TokenGen $O(1)$ Distance $O(1)$
Our VPCS	double-CSP	GraphEnc $O(n^2)$ TokenGen $O(1)$ CSPQuery $O(n)$	TokenGen $O(1)$ Distance $O(1)$



(a) (b) (c) (d)

Fig. 2: Query time. (a)ego-facebook. (b)p2p-Gnutella06. (c)wiki-vote. (d)email-eu-core.



(a) (b) (c) (d)

Fig. 3: Verify time. (a)ego-facebook. (b)p2p-Gnutella06. (c)wiki-vote. (d)email-eu-core.

TABLE III: Detailed Information of Different Graph Datasets

Dataset	Vertices	Edges	Storage
ego-Facebook	4,039	88,234	834 KB
p2p-Gnutella06	8,717	31,525	325 KB
wiki-Vote	7,115	103,689	1.04 MB
email-Eu-core	1,005	25,571	77.9 KB

allowing the cloud server to filter them directly. However, for every edge, whether dummy or real, SPCS executes two comparison protocols and its SXD protocol, resulting in a slightly shorter time compared to VPCS-II. However, this also leads to the leakage of the topological structure of the graph data owned by the GO.

Therefore, our scheme sacrifices an acceptable amount of time to support higher security and privacy protection for outsourced data.

Fig. 3 displays the execution time of our VPCS scheme for performing verifiable double-CSD queries on different datasets. The ”proof generation” indicates the time taken by the cloud server to generate verification proofs, which clearly increases gradually with the size of the queried graph. In contrast, the “verify’ represents the time required by the query user(QU) to validate the query results, and it shows no correlation with the size of the queried graph.

Overall, the experimental results illustrate that our verifiable CSD query scheme can enhance the security of outsourced graph data computation while moderately sacrificing efficiency. Furthermore, it is the first strategy to support verifiable CSD queries.

VIII. CONCLUSION

In this study, we present a novel scheme called VPCS that supports verifiable double-CSP queries. To the best of our knowledge, this is the first scheme of its kind. We have confirmed that VPCS can verify queries without disclosing any information about other graph data. Moreover, VPCS achieves IND-CQVA-security and demonstrates acceptable efficiency when applied to real-world datasets. Our future work will

involve exploring additional strategies for verifiable graph privacy-preserving queries.

REFERENCES

[1] S. Ji, P. Mittal, and R. Beyah, "Graph data anonymization, de-anonymization attacks, and de-anonymizability quantification: A survey," *IEEE Communications Surveys & Tutorials*, vol. 19, no. 2, pp. 1305–1326, 2016.

[2] J. Qian, X.-Y. Li, C. Zhang, L. Chen, T. Jung, and J. Han, "Social network de-anonymization and privacy inference with knowledge graph model," *IEEE Transactions on Dependable and Secure Computing*, vol. 16, no. 4, pp. 679–692, 2017.

[3] Y. Low, J. E. Gonzalez, A. Kyrola, D. Bickson, C. E. Guestrin, and J. Hellerstein, "Graphlab: A new framework for parallel machine learning," *arXiv preprint arXiv:1408.2041*, 2014.

[4] S. Ko and W.-S. Han, "Turbograph++ a scalable and fast graph analytics system," in *Proceedings of the 2018 International Conference on Management of Data*, pp. 395–410, 2018.

[5] R. K. Ibrahim, S. R. Zeebaree, K. Jacksi, S. H. Ahmed, S. M. Mohammed, R. R. Zebari, A. Alkhayyat, and Z. N. Rashid, "Clustering document based on semantic similarity using graph base spectral algorithm," in *2022 5th International Conference on Engineering Technology and its Applications (IICETA)*, pp. 254–259, IEEE, 2022.

[6] S. Storandt, "Route planning for bicycles—exact constrained shortest paths made practical via contraction hierarchy," in *Proceedings of the International Conference on Automated Planning and Scheduling*, vol. 22, pp. 234–242, 2012.

[7] S. Wang, X. Xiao, Y. Yang, and W. Lin, "Effective indexing for approximate constrained shortest path queries on large road networks," *Proceedings of the VLDB Endowment*, vol. 10, no. 2, pp. 61–72, 2016.

[8] G. Tsaggouris and C. Zaroliagis, "Multiobjective optimization: Improved fptas for shortest paths and non-linear objectives with applications," *Theory of Computing Systems*, vol. 45, no. 1, pp. 162–186, 2009.

[9] X. Meng, S. Kamara, K. Nissim, and G. Kollios, "Grecs: Graph encryption for approximate shortest distance queries," in *Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security*, pp. 504–517, 2015.

[10] C. Zhang, L. Zhu, C. Xu, K. Sharif, C. Zhang, and X. Liu, "Pgas: Privacy-preserving graph encryption for accurate constrained shortest distance queries," *Information Sciences*, vol. 506, pp. 325–345, 2020.

[11] W. Wang, Z. Jia, M. Xu, and S. Li, "Spcs: Strong privacy-preserving-constrained shortest distance queries on encrypted graphs," *IEEE Internet of Things Journal*, vol. 9, no. 22, pp. 22516–22528, 2022.

[12] X. Yao, R. Zhang, D. Huang, and Y. Zhang, "Verifiable query processing over outsourced social graph," *IEEE/ACM Transactions on Networking*, vol. 29, no. 5, pp. 2313–2326, 2021.

[13] X. Li, C. Weng, Y. Xu, X. Wang, and J. Rogers, "Zksql: Verifiable and efficient query evaluation with zero-knowledge proofs," *Proceedings of the VLDB Endowment*, vol. 16, no. 8, pp. 1804–1816, 2023.

[14] A. Viand, C. Knabenhans, and A. Hithnawi, "Verifiable fully homomorphic encryption," *arXiv preprint arXiv:2301.07041*, 2023.

[15] P. Ghosh and V. Shah, "New lower bounds in merlin-arthur communication and graph streaming verification," *arXiv preprint arXiv:2401.06378*, 2024.

[16] Y. Peng, Z. Fan, B. Choi, J. Xu, and S. S. Bhowmick, "Authenticated subgraph similarity searchin outsourced graph databases," *IEEE Transactions on Knowledge and Data Engineering*, vol. 27, no. 7, pp. 1838–1860, 2014.

[17] Z. Fan, Y. Peng, B. Choi, J. Xu, and S. S. Bhowmick, "Towards efficient authenticated subgraph query service in outsourced graph databases," *IEEE Transactions on Services Computing*, vol. 7, no. 4, pp. 696–713, 2013.

[18] H. Chen, Q. Qu, Y. Lin, X. Chen, and K. Li, "Authenticity verification on social data outsourcing," *Computers & Security*, vol. 100, p. 102077, 2021.

[19] Z. Chang, L. Zou, and F. Li, "Privacy preserving subgraph matching on large graphs in cloud," in *Proceedings of the 2016 International Conference on Management of Data*, pp. 199–213, 2016.

[20] Z. Xu, F. Zhou, Y. Li, J. Xu, and Q. Wang, "Privacy-preserving subgraph matching protocol for two parties," *International Journal of Foundations of Computer Science*, vol. 30, no. 04, pp. 571–588, 2019.

[21] G. Suntaxi, A. A. El Ghazi, and K. Böhm, "Secrecy and performance models for query processing on outsourced graph data," *Distributed and Parallel Databases*, vol. 39, pp. 35–77, 2021.

[22] X. Liu, X.-F. Tu, D. Luo, G. Xu, N. N. Xiong, and X.-B. Chen, "Secure multi-party computation of graphs' intersection and union under the malicious model," *Electronics*, vol. 12, no. 2, p. 258, 2023.

[23] M. Chase and S. Kamara, "Structured encryption and controlled disclosure," in *Advances in Cryptology-ASIACRYPT 2010: 16th International Conference on the Theory and Application of Cryptology and Information Security, Singapore, December 5-9, 2010. Proceedings 16*, pp. 577–594, Springer, 2010.

[24] M. Blanton, A. Steele, and M. Alisagari, "Data-oblivious graph algorithms for secure computation and outsourcing," in *Proceedings of the 8th ACM SIGSAC symposium on Information, computer and communications security*, pp. 207–218, 2013.

[25] A. Aly, E. Cuvelier, S. Mawet, O. Pereira, and M. Van Vyve, "Securely solving simple combinatorial graph problems," in *Financial Cryptography and Data Security: 17th International Conference, FC 2013, Okinawa, Japan, April 1-5, 2013, Revised Selected Papers 17*, pp. 239–257, Springer, 2013.

[26] C. Liu, L. Zhu, X. He, and J. Chen, "Enabling privacy-preserving shortest distance queries on encrypted graph data," *IEEE Transactions on Dependable and Secure Computing*, vol. 18, no. 1, pp. 192–204, 2018.

[27] H. Zhu, B. Wu, M. Xie, Z. Cui, and Z. Wu, "Secure shortest path search over encrypted graph supporting synonym query in cloud computing," in *2016 IEEE Trustcom/BigDataSE/ISPA*, pp. 497–504, IEEE, 2016.

[28] Z. Lu, Y. Feng, and Q. Cao, "Decentralized search for shortest path approximation in large-scale complex networks," in *2017 IEEE International Conference on Cloud Computing Technology and Science (CloudCom)*, pp. 130–137, IEEE, 2017.

[29] M. Shen, B. Ma, L. Zhu, R. Mijumbi, X. Du, and J. Hu, "Cloud-based approximate constrained shortest distance queries over encrypted graphs with privacy protection," *IEEE Transactions on Information Forensics and Security*, vol. 13, no. 4, pp. 940–953, 2017.

[30] M. Shen, S. Chen, L. Zhu, R. Xiao, K. Xu, and X. Du, "Privacy-preserving graph encryption for approximate constrained shortest distance queries," in *2019 IEEE Global Communications Conference (GLOBECOM)*, pp. 1–6, IEEE, 2019.

[31] R. Gennaro, C. Gentry, and B. Parno, "Non-interactive verifiable computing: Outsourcing computation to untrusted workers," in *Advances in Cryptology–CRYPTO 2010: 30th Annual Cryptology Conference, Santa Barbara, CA, USA, August 15-19, 2010. Proceedings 30*, pp. 465–482, Springer, 2010.

[32] Y. Zhu, H. Li, J. Cui, and Y. Ma, "Verifiable subgraph matching with cryptographic accumulators in cloud computing," *IEEE Access*, vol. 7, pp. 169636–169645, 2019.

[33] X. Zuo, L. Li, S. Luo, H. Peng, Y. Yang, and L. Gong, "Privacy-preserving verifiable graph intersection scheme with cryptographic accumulators in social networks," *IEEE Internet of Things Journal*, vol. 8, no. 6, pp. 4590–4603, 2020.

[34] X. Liu, K.-K. R. Choo, R. H. Deng, R. Lu, and J. Weng, "Efficient and privacy-preserving outsourced calculation of rational numbers," *IEEE Transactions on Dependable and Secure Computing*, vol. 15, no. 1, pp. 27–39, 2016.